Introduction to Machine Learning

Lecture 11: Generalization and Gradient Descent

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Winter 2026

Quick Recap: *Unsupervised and Supervised Learning*

In Unsupervised Learning, samples are unlabeled

- *Data* $\rightsquigarrow$ *Collection of samples* $\mathbb{D} = \{\boldsymbol{x}_n : n = 1, \ldots, N\}$
- *Model* $\rightsquigarrow$ *Captures a pattern observed in data, e.g., fitting into clusters*
- *Learning algorithm* $\rightsquigarrow$ *It takes* $\mathbb{D}$ *and returns a good model*

In Supervised Learning, samples are labeled

- *Data* $\rightsquigarrow$ *Collection of samples* $\mathbb{D} = \{(\boldsymbol{x}_n, \boldsymbol{v}_n) : n = 1, \ldots, N\}$
- *Model* $\rightsquigarrow$ *Captures relation between data samples and their labels*
- *Learning algorithm* $\rightsquigarrow$ *It takes* $\mathbb{D}$ *and returns a good model*

Quick Recap: *Unsupervised Learning*

- *We assume samples are coming i.i.d from a distribution*
- *We model the distribution via a parameterized model*

$$P_{\boldsymbol{\theta}}\left(\boldsymbol{x}\right)$$

- *We learn the parameters via maximum likelihood*

$$\max_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^{N} \log P_{\boldsymbol{\theta}}\left(\boldsymbol{x}_n\right)$$

We looked into

- Clustering $\rightsquigarrow$ K-means Clustering
- Density learning
- Dimensionality reduction $\rightsquigarrow$ PCA

Quick Recap: *Supervised Learning*

- *Labeled dataset*

$$\mathbb{D} = \{(\boldsymbol{x}_n \in \mathbb{X}, \boldsymbol{v}_n \in \mathbb{V}) : n = 1, \ldots, N\}$$

- *Model from hypothesis set $\mathbb{H}$ that relates samples to labels*

$$f : \mathbb{X} \mapsto \mathbb{V}$$

- *Learning algorithm finds optimal model within hypothesis set*

$$\mathcal{A} : \mathbb{D} \mapsto f^\star \in \mathbb{H}$$

Quick Recap: *Regression versus Classification*

We first focused on linear models, i.e.,

$$f(\boldsymbol{x}) = \mathbf{w}^{\mathsf{T}}\boldsymbol{x} \rightsquigarrow \begin{cases} y = f(\boldsymbol{x}) & \text{Regression} \\ y = \operatorname{sign}(f(\boldsymbol{x})) & \text{Classification} \end{cases}$$

But, we could go beyond linear models via SVM, where

$$f(\boldsymbol{x}) = \mathbf{w}^{\mathsf{T}}\varphi(\boldsymbol{x})$$

for some embedding $\varphi(\boldsymbol{x})$

- *If we use SVC, we don't need $\varphi(\boldsymbol{x})$*
- *We can work with the kernel $\rightsquigarrow$ kernel trick*

Today's Agenda: *Generalization and Gradient Descent*

Today, we discuss two key concepts that enable us test and train

Generalization and *Gradient Descent Algorithm*

The concepts are discussed as follows

- *How to test a learned model*
  - ↳ *Generalization*
- *How to minimize a function iteratively*
  - ↳ *Gradient Descent Algorithm*

Testing A Model

- ? *How can we know if our model has learned the pattern?*
- ! *We can test it*

Let's think about both cases of

- *unsupervised learning*
- *supervised learning*

Testing A Model : *Unsupervised*

In unsupervised learning we maximize the likelihood

$$\mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}^{\star}\right) = \max_{\boldsymbol{\theta}} \mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}\right)$$

Collect a new set of samples

$$\mathbb{T} = \{\boldsymbol{x}_i : i = 1, \ldots, I\}$$

Use the trained model $\boldsymbol{\theta}^{\star}$ to find the likelihood

$$\mathcal{S}_{\mathbb{T}}\left(\boldsymbol{\theta}^{\star}\right) = \log P_{\boldsymbol{\theta}^{\star}}\left(\mathbb{T}\right)$$

- *If* $\mathcal{S}_{\mathbb{T}}\left(\boldsymbol{\theta}^{\star}\right) \approx \mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}^{\star}\right) \rightsquigarrow$ *the model generalizes*
- *If* $\mathcal{S}_{\mathbb{T}}\left(\boldsymbol{\theta}^{\star}\right) \ll \mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}^{\star}\right) \rightsquigarrow$ *the model does not generalize!*
  - *Maybe we need more complicated or simpler model!*

Validating the Model

We typically reserve some samples to validate

$$\mathbb{V} = \{\boldsymbol{x}_j : i = 1, \ldots, J\}$$

We train the model $\boldsymbol{\theta}^\star$ on $\mathbb{D}$; then, check

$$\mathcal{S}_{\mathbb{V}}\left(\boldsymbol{\theta}^\star\right) = \log P_{\boldsymbol{\theta}^\star}\left(\mathbb{V}\right)$$

- *If* $\mathcal{S}_{\mathbb{V}}\left(\boldsymbol{\theta}^\star\right) \approx \mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}^\star\right) \rightsquigarrow$ *the model is valid*
  - *We now check generalization on the test set* $\mathbb{T}$
- *If* $\mathcal{S}_{\mathbb{V}}\left(\boldsymbol{\theta}^\star\right) \ll \mathcal{S}_{\mathbb{D}}\left(\boldsymbol{\theta}^\star\right) \rightsquigarrow$ *the model is not valid!*
  - *We change some hyperparameters in the model and repeat*

Example: K-means Clustering

Consider clustering problem

1. *Choose some hyperparameter K*
2. *Train via K-means clustering on $\mathbb{D}$*
   - ↳ *Find $\mu_1, \ldots, \mu_K$ and save the final risk*

$$\mathcal{J}_{\text{train}} = \frac{1}{N} \sum_{k=1}^{K} \sum_{n=1}^{N} r_{n,k} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

3. *Compute the risk on the set $\mathbb{V}$ and call it $\mathcal{J}_{\text{val}}$*
   - ↳ *If $\mathcal{J}_{\text{val}} \approx \mathcal{J}_{\text{train}} \rightsquigarrow$ the model is valid*
     - ↳ *We now test if $\mathcal{J}_{\text{test}}$ on $\mathbb{T}$ is also close*
   - ↳ *If $\mathcal{J}_{\text{val}} \gg \mathcal{J}_{\text{train}} \rightsquigarrow$ the model is not valid!*
     - ↳ *We change K to higher/lower value and get back to (1)*

Testing A Model : *Supervised*

Supervised learning we minimize risk

$$\hat{R}_{\mathbb{D}}\left(\mathbf{w}^{\star}\right) = \min_{\mathbf{w}} \mathbb{E}_{\mathbb{D}}\left\{\mathcal{L}\left(y, v\right)\right\}$$

Collect a new set of samples

$$\mathbb{T} = \left\{\left(\boldsymbol{x}_i, v_i\right) : i = 1, \ldots, I\right\}$$

Use the trained model to estimate risk on test set

$$\hat{R}_{\mathbb{T}}\left(\mathbf{w}^{\star}\right) = \min_{\mathbf{w}} \mathbb{E}_{\mathbb{T}}\left\{\mathcal{L}\left(y, v\right)\right\}$$

- *If* $\hat{R}_{\mathbb{T}}\left(\mathbf{w}^{\star}\right) \approx \hat{R}_{\mathbb{D}}\left(\mathbf{w}^{\star}\right) \rightsquigarrow$ *the model generalizes*
- *If* $\hat{R}_{\mathbb{T}}\left(\mathbf{w}^{\star}\right) \gg \hat{R}_{\mathbb{D}}\left(\mathbf{w}^{\star}\right) \rightsquigarrow$ *the model does not generalize!*
  - *Maybe we need more complicated or simpler model!*

Example: *Soft SVC*

Consider soft SVC

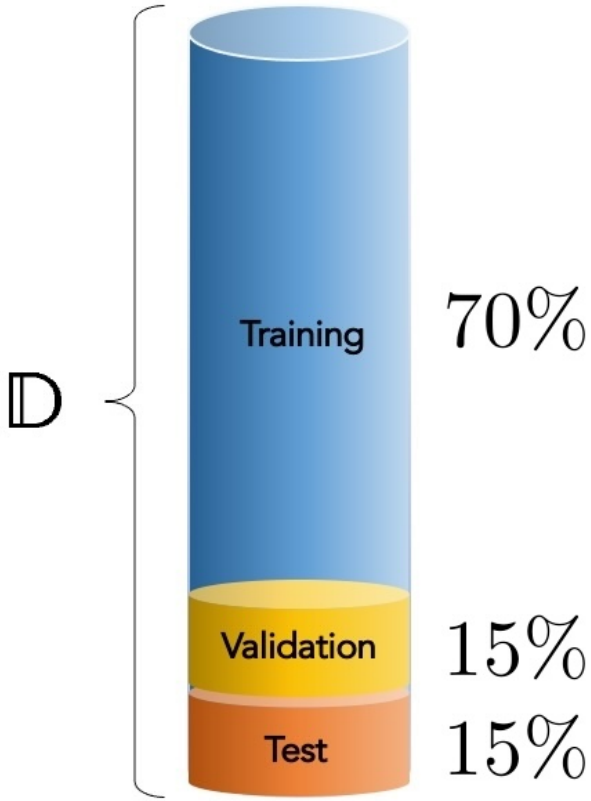
1. *Choose some parameter C*
2. *Train on $\mathbb{D}$*

$$\min_{\mathbf{w},\xi_n \geqslant 0} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{n=1}^{N} \xi_n \qquad \textit{subject to } \; v_n \mathbf{w}^\mathsf{T} \boldsymbol{x}_n \geqslant 1 - \xi_n \textit{ for all } n \in \mathbb{D}$$

3. *Estimate error rate E on the sets $\mathbb{V}$ and $\mathbb{D}$*
   - ↳ *If $\hat{E}_\mathbb{V} \approx \hat{E}_\mathbb{D}$ $\rightsquigarrow$ the model is valid*
     - ↳ *We now test if error rate on $\mathbb{T}$ is also close*
   - ↳ *If $\hat{E}_\mathbb{V} \gg \hat{E}_\mathbb{D}$ $\rightsquigarrow$ the model is not valid!*
     - ↳ *We change C to higher/lower value and get back to (1)*

Training vs Test Data

In practice, we split our collected data



Further Read

- Bishop
  - ↳ Chapter 1: *Section 1.3* — ***Validation and Test***
- Goodfellow
  - ↳ Chapter 5: *Sections 5.2 and 5.3* — ***Generalization***

Approximating Optimal Model $\equiv$ Training

Except linear regression, none of optimal models is explicitly determined

- *There might be a unique minimizer, but not explicit*
  - ↳ *Logistic regression*
- *It might be an enormous number of minimizers*
  - ↳ *Almost all neural networks as we will see!*

We would be hence OK to find a rather fair local minimizer

Training

The procedure of finding a minimizer for the empirical risk is called training

Algorithmic Approach to Training

In practice, we use algorithmic approaches: *say the model depends on* $\mathbf{w}$

```
GenericTraining():
 1: Start with w
 2: while w not converged do
 3:     Update w
 4: end while
 5: Return w
```

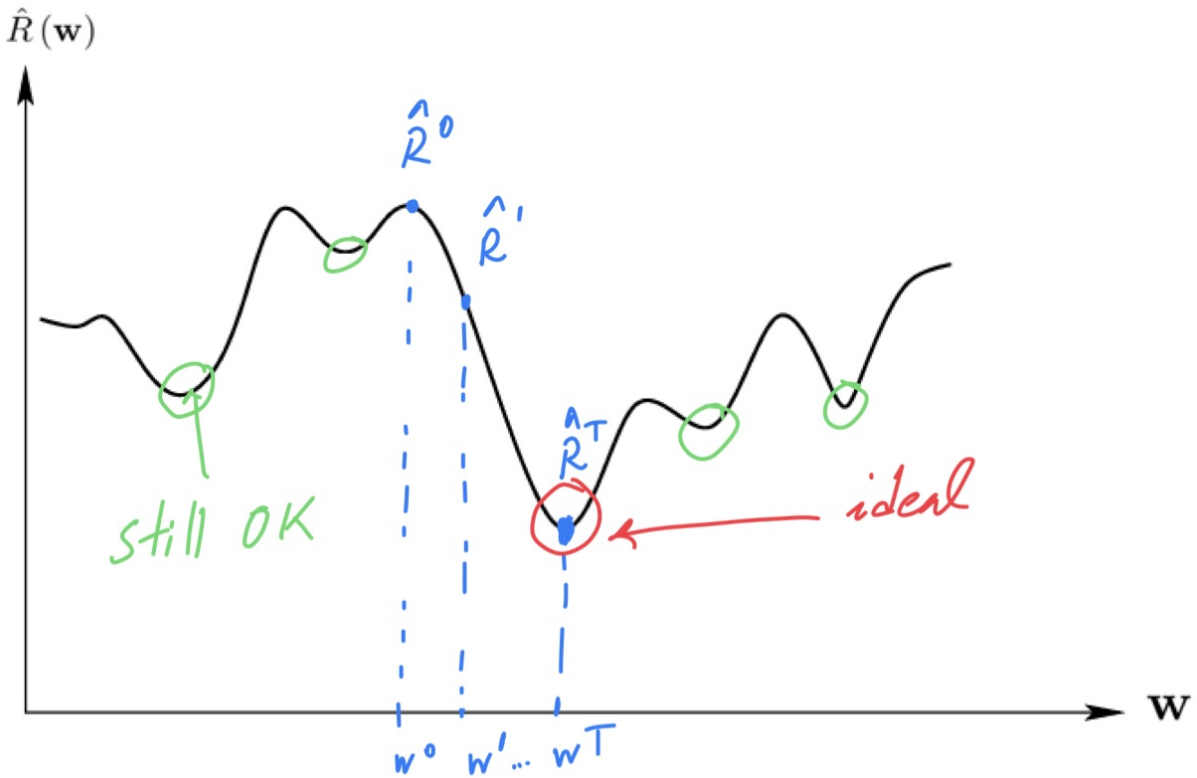
? *What is the right approach to update?*

Optimization Algorithm $\equiv$ *Optimizer*

Let's look at it from a more generic perspective: *we deal with an optimization*

$$\min_{\mathbf{w}} \hat{R}(\mathbf{w})$$

and want to design an optimizer



General Optimizer

`Optimizer()`:

1: Choose some threshold ϵ and step size η
2: Start with an arbitrary $\mathbf{w}^0$
3: **while** $t < 1$ or $\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2 > \epsilon$ **do**
4: Compute a moving direction

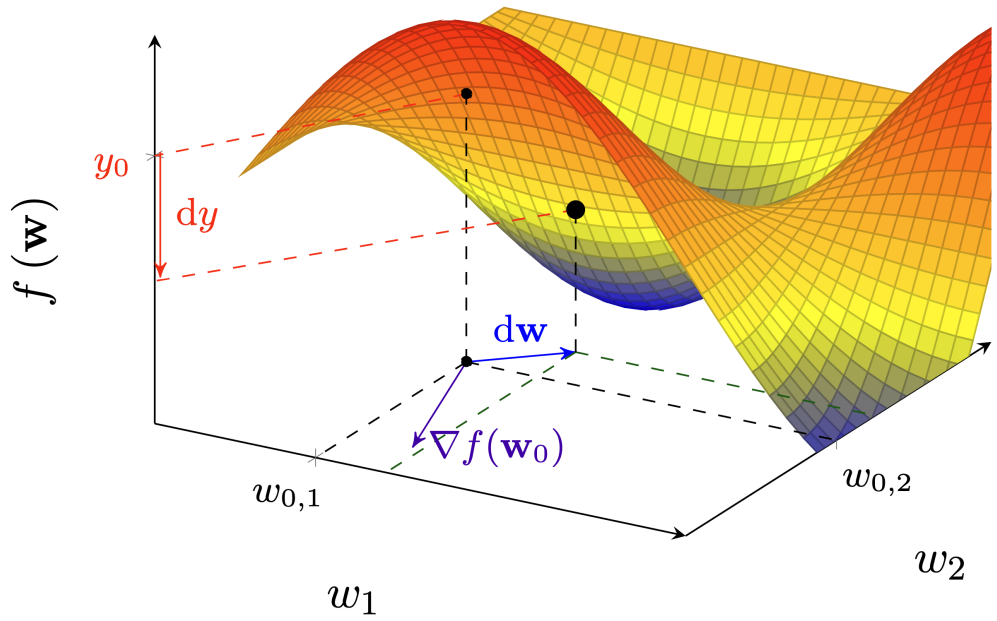
$$\boldsymbol{\mu}^t \leftarrow \texttt{direction.calculator}\left(\hat{R}, \mathbf{w}^t\right)$$

5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t + \eta\boldsymbol{\mu}^t$
6: Update $t \leftarrow t + 1$
7: **end while**
8: Return final $\mathbf{w}^t$

We need to find a systematic approach to compute a good direction $\boldsymbol{\mu}$

? *What is a good direction?*

Gradients: *Extension to Higher Dimensions*



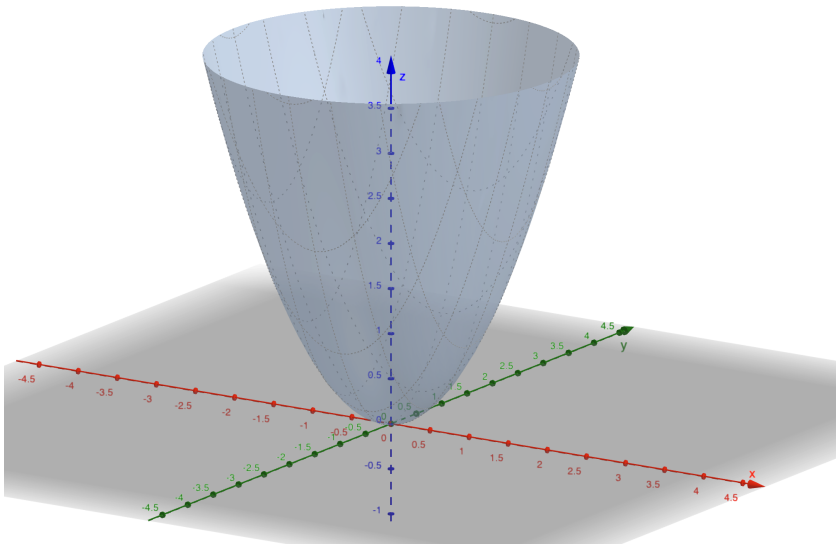
Gradient of a function shows a key property

- *its magnitude gives the slope of tangent hyperplane*
- *its direction tells us where to move to increase maximally*
  - *its negative direction tells us where to move to decrease maximally*

Example: *3D Paraboloid*

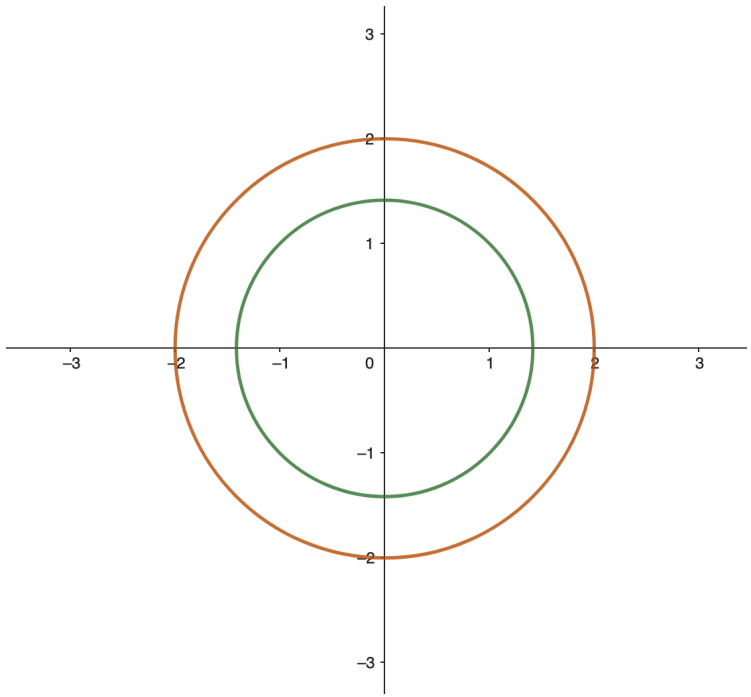
The surface is

$$f(\mathbf{w}) = w_1^2 + w_2^2$$



We can also plot the counters

$$f(\mathbf{w}) = \alpha$$

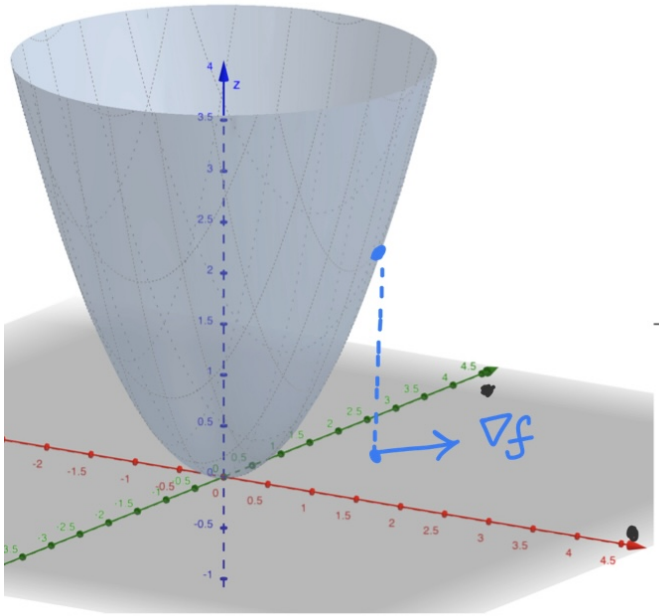


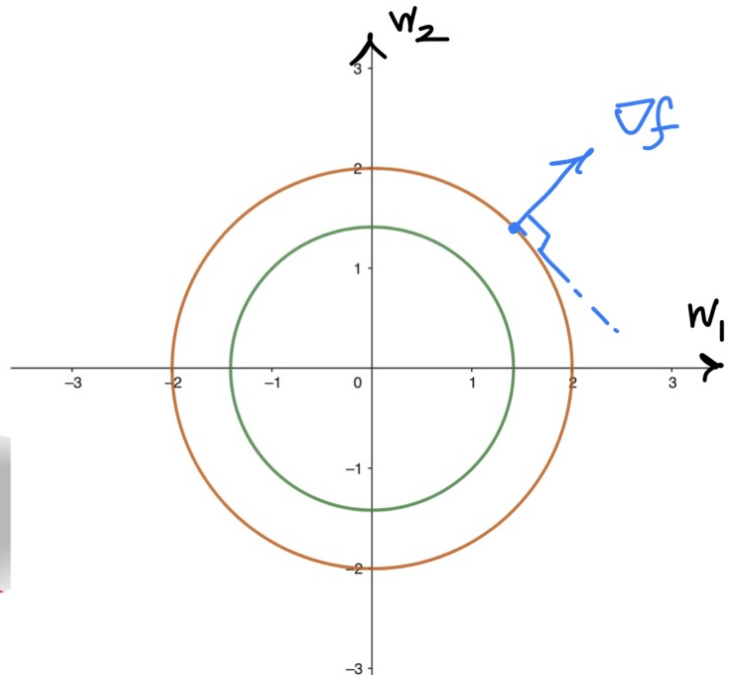
Example: *3D Paraboloid*

Let's compute the gradient

$$f(\mathbf{w}) = w_1^2 + w_2^2 \rightsquigarrow \nabla f = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}$$

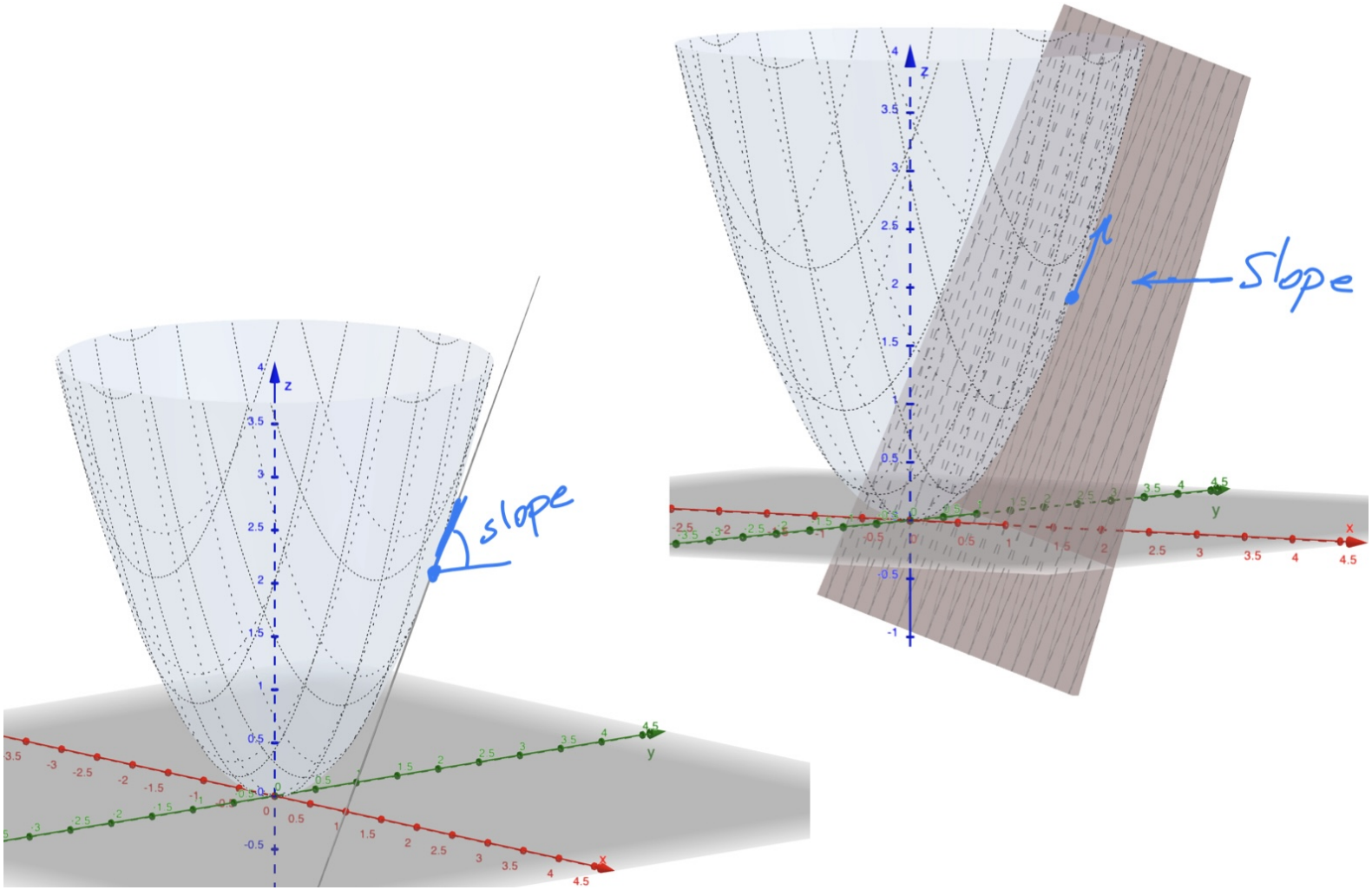
It points to the direction that we increase maximally





Example: *3D Paraboloid*

The amplitude is proportional to the slope of touching plane



Gradient Direction $\equiv$ *Steepest Direction*

Moral of Story

Gradient direction is the steepest direction on the surface

Optimizer looks for a direction that makes the function smaller: *we could move in the negative direction of gradient*

$$\boldsymbol{\mu}^t \leftarrow \texttt{direction.calculator}\left(\hat{R}, \mathbf{w}^t\right) = -\nabla \hat{R}\left(\mathbf{w}^t\right)$$

This means that we update

$$\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}\left(\mathbf{w}^t\right)$$

? *Does it end at a minimum?*

Moving with Gradient

Optimizer stops when

$$\|\mathbf{w}^{t+1} - \mathbf{w}^{t}\|^2 \leqslant \epsilon$$

As we move with gradient, we have at the stop point

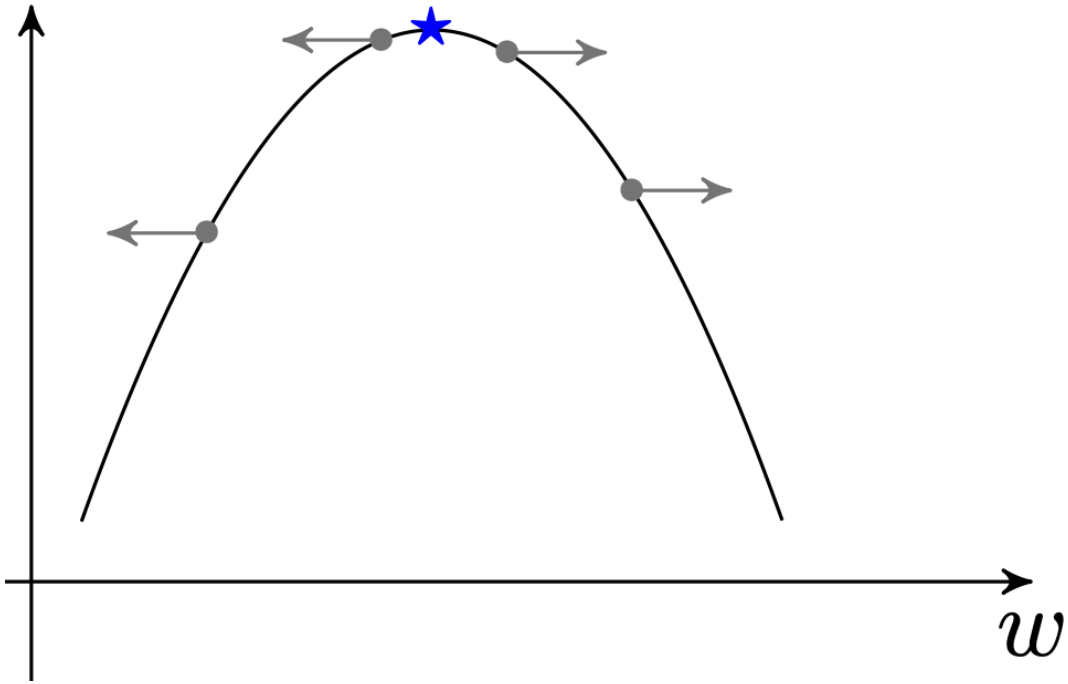
$$\|\mathbf{w}^{t+1} - \mathbf{w}^{t}\|^2 = \|\nabla \hat{R}\left(\mathbf{w}^{t}\right)\|^2 \approx 0$$

So, we stop when $\nabla \hat{R}\left(\mathbf{w}^{t}\right) \approx \mathbf{0}$ which is

- *Minimum*
- *Maximum*
- *Saddle Point*

Moving with Gradient

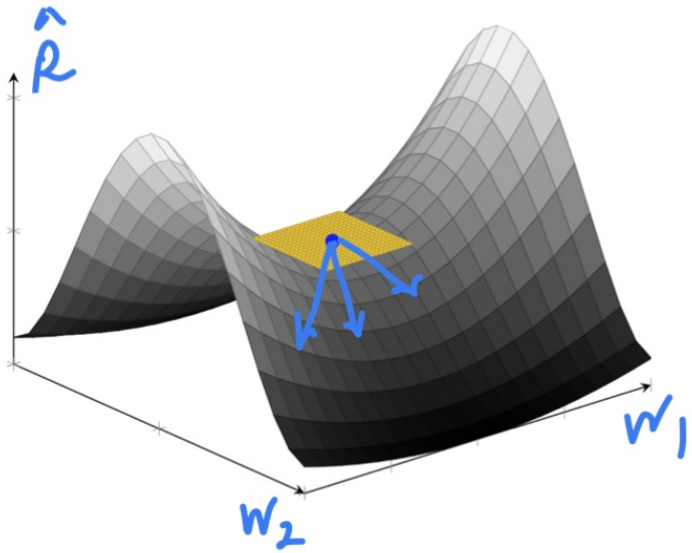
? *Can it get trapped at a maximum?*

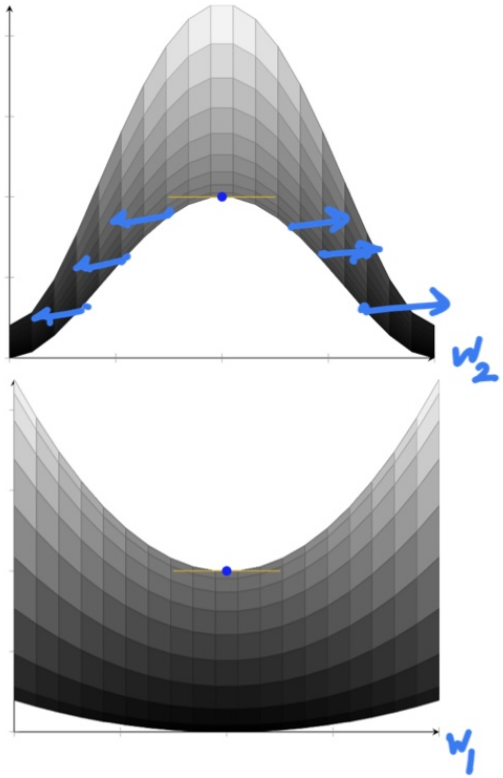


! *No!*

Moving with Gradient

? *Can it get trapped at a saddle point?*

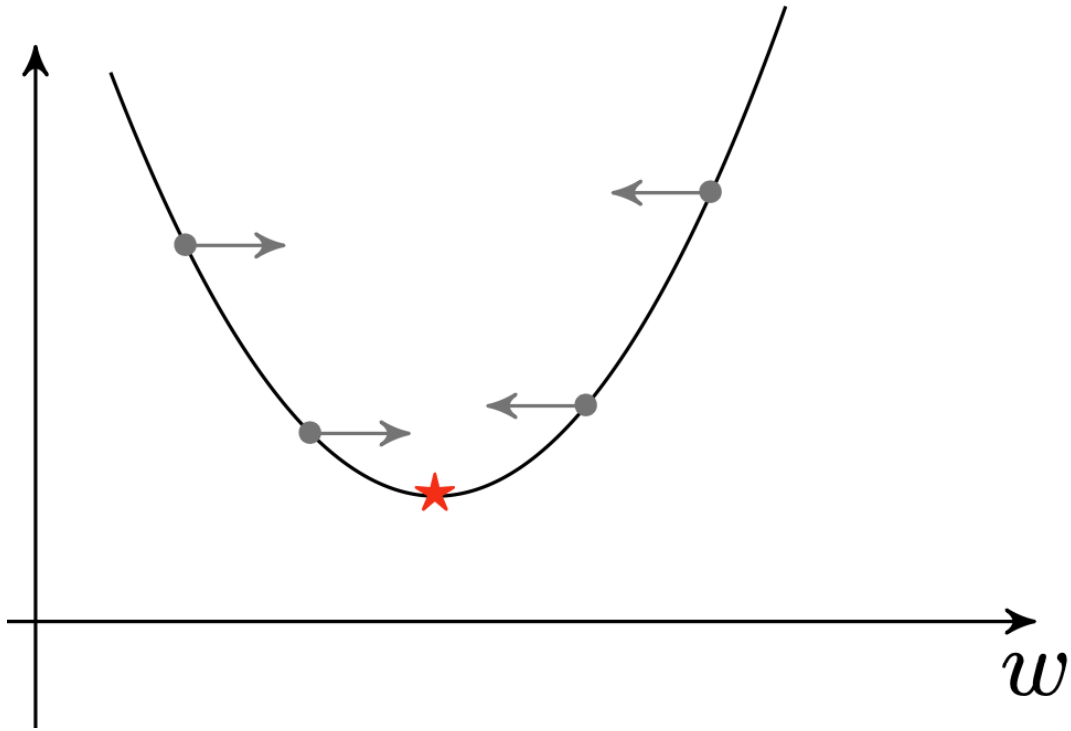




! *No!*

Moving with Gradient

? *Can it get trapped at a minimum?*



! *Yes!*

Gradient Descent Algorithm

`GradientDescent():`

1: Choose some threshold ϵ and step size η
2: Start with an arbitrary $\mathbf{w}^0$
3: **while** $t < 1$ or $\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2 > \epsilon$ **do**
4: Compute the gradient at $\mathbf{w}^t$
5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}\left(\mathbf{w}^t\right)$
6: Update $t \leftarrow t + 1$
7: **end while**
8: Return final $\mathbf{w}^t$

The algorithm starts at $\mathbf{w}^0$ and moves to a local minimum at step η

↳ *η is what we call learning rate*

Generic Form: *Gradient-based Optimizer*

You might hear the term

Gradient-based Optimizer

This is pretty much the same thing

`GradientOptim()`:

1: Choose some threshold ϵ and step size η
2: Start with an arbitrary $\mathbf{w}^0$
3: **while** $t < 1$ or $\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2 > \epsilon$ **do**
4: Compute the gradient at $\mathbf{w}^t$
5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta f_\beta \left(\nabla \hat{R}\left(\mathbf{w}^t\right)\right)$
6: Update $t \leftarrow t + 1$
7: **end while**
8: Return final $\mathbf{w}^t$

$f_\beta(\cdot)$ *is an operation usually some linear operation*

Example: *Linear Regression*

In linear regression, the empirical risk is

$$\hat{R}(\mathbf{w}) = \frac{1}{N}\|\mathbf{X}^\mathsf{T}\mathbf{w} - \boldsymbol{v}\|^2$$

At point $\mathbf{w}^t$, we have

$$\nabla\hat{R}(\mathbf{w}^t) = \frac{2}{N}\left(\mathbf{X}\mathbf{X}^\mathsf{T}\mathbf{w}^t - \mathbf{X}\boldsymbol{v}\right)$$

We then update as

$$\begin{aligned}\mathbf{w}^{t+1} &\leftarrow \mathbf{w}^t - \frac{2\eta}{N}\left(\mathbf{X}\mathbf{X}^\mathsf{T}\mathbf{w}^t - \mathbf{X}\boldsymbol{v}\right)\\ &\leftarrow \left(\mathbf{I}_d - \frac{2\eta}{N}\mathbf{X}\mathbf{X}^\mathsf{T}\right)\mathbf{w}^t + \frac{2\eta}{N}\mathbf{X}\boldsymbol{v}\end{aligned}$$

Example: *Linear Regression*

? *When does it stop?*

When $\mathbf{w}^{t+1} \approx \mathbf{w}^t$, i.e.,

$$\mathbf{w}^t \approx \mathbf{w}^t - \frac{2\eta}{N}\left(\mathbf{X}\mathbf{X}^\mathsf{T}\mathbf{w}^t - \mathbf{X}\boldsymbol{y}\right)$$

or equivalently

$$\mathbf{X}\mathbf{X}^\mathsf{T}\mathbf{w}^t - \mathbf{X}\boldsymbol{y} \approx 0 \rightsquigarrow \mathbf{w}^t \approx \left(\mathbf{X}\mathbf{X}^\mathsf{T}\right)^{-1}\mathbf{X}\boldsymbol{y}$$

! *Bingo! Same as the analytic result!*

Convergence Guarantee

Gradient descent is guaranteed to end to a local minimum

- *Global convergence is only guaranteed if empirical risk is convex*
  - ↳ *Linear and logistic regression*
- *Learning rate is crucial in the behavior*
  - ↳ *With very large learning rates it can diverge*
  - ↳ *With very small learning rates it takes long time*
  - ↳ *With very small learning rates we may get trapped in bad local minimum*

Further Read

- Goodfellow
  - ↳ Chapter 4: *Section 4.3 – 4.5* ***Gradient Descent***
- Bishop
  - ↳ Chapter 5: *Section 5.2* ***Optimizers***
- ESL
  - ↳ Chapter 10: *Section 10.10* ***Numerical Optimization***